AI Applications in Research:

Large Language Model Agents and Claude Code in Empirical Workflows

Prepared from *Scott's Mixtape Substack* (Scott Cunningham)
and the broader "Claude Code for quantitative social scientists" series

June 15, 2026

Abstract

This note summarizes the main concepts surrounding the use of artificial intelligence—and in particular agentic coding assistants such as **Claude Code**—in academic and applied research. It is organized around the themes that recur in Scott Cunningham's ongoing newsletter series on AI agents for quantitative social science: how agents change the research production function, the practical workflow of using a coding agent for applied econometrics, the failure modes that demand discipline (context loss, fabrication, reproducibility), and the emerging norms of transparency and disclosure. The aim is conceptual orientation rather than a tool tutorial.

Contents

1 What Claude Code Is **1**

2 How AI Changes the Research Production Function **1**

3 A Workflow for Applied Econometrics **2**

4 Failure Modes and How to Manage Them **2**

5 Transparency, Disclosure, and Research Norms **3**

6 Open Research on the Economics of AI **3**

7 Takeaways **3**

1 What Claude Code Is

Claude Code is a command-line "agentic" coding tool built on Anthropic's Claude models. Unlike a chat window that returns text, an agent operates *inside* a working environment: it can read and write files, run shell commands, execute scripts, inspect their output, and iterate—taking multiple autonomous steps toward a stated goal. For a researcher this means the assistant is not merely suggesting code but actually running the analysis pipeline: loading data, estimating models, reading error messages, correcting itself, and producing tables and figures.

- **Agentic, not conversational.** It plans, acts, observes the result, and re-plans—a loop, not a single answer.

- **Tool-using.** It has access to the file system, a terminal, version control, and can be extended with custom tools and external services (e.g. via the Model Context Protocol).
- **Repository-aware.** It builds context from the actual project on disk, so its suggestions are grounded in the real code and data rather than in a generic example.

2 How AI Changes the Research Production Function

A recurring theme is that AI agents shift *where* a researcher's time goes rather than simply "doing the work." The bottleneck moves from mechanical implementation toward specification, judgment, and verification.

Lowering the cost of execution.
Tasks that used to "get stuck" in a researcher's pipeline—data cleaning, merging unfamiliar datasets, translating an estimator from one language to another, building a robustness table—become cheap to attempt. Projects that stalled for years for want of execution time can be revived.

Raising the value of clear thinking.
The agent will do whatever it is asked; the binding constraint becomes the precision of the research question, the identification strategy, and the specification. Vague instructions yield confident but wrong output.

Compressing the learning loop.
A researcher can ask an agent to implement an estimator, then explain the derivation, then stress-test it on simulated data—turning method acquisition into an interactive, hands-on exercise.

3 A Workflow for Applied Econometrics

The series repeatedly stresses that productive use of an agent in empirical work is *structured*, not improvised. A representative workflow:

1. **State the design first.** Write down the estimand, the identification assumption (e.g. parallel trends for difference-in-differences), and the sample before writing code.
2. **Decompose into small, checkable steps.** Have the agent build the pipeline incrementally—ingest, clean, describe, estimate, validate—so each stage can be inspected.
3. **Simulate before you trust.** Use synthetic data with a known ground truth to confirm the estimator recovers the true parameter before pointing it at real data.
4. **Demand the derivation, not just the number.** Ask the agent to explain *why* a specification is correct, so errors of reasoning surface.
5. **Verify against an independent path.** Cross-check key results with a second implementation or a hand calculation.

The case for a detailed checklist

A central argument of the series is that applied econometrics with an agent *requires a detailed checklist*. Because the agent is fast and fluent, it is easy to accept output that is subtly wrong—wrong cluster level, covariates that fix levels but not trends, an estimator silently dropping observations. A written checklist (sample construction, treatment timing, clustering, balance, parallel-trends diagnostics, weighting) turns implicit expertise into an explicit gate that every analysis must pass.

4 Failure Modes and How to Manage Them

"Amnesia" / context limits.
An agent's working memory is finite. Over a long session it loses track of earlier decisions, reintroduces fixed bugs, or contradicts prior choices. The remedy is externalizing memory: persistent project notes, a written specification, and committing state to version control so the record lives outside the model's context window.

Confident fabrication.
Models can produce plausible-but-false statistics, citations, or code that runs yet computes the wrong thing. Treat every output as a claim to be verified, not a fact.

Reproducibility.
Non-deterministic generation threatens replication. Defenses: pin seeds, save the exact scripts the agent produced (not just the answers), version everything, and keep a human-readable log of the analytical decisions.

Selection and data-quality blind spots.
The agent will faithfully analyze a biased sample without flagging the bias—e.g. a survey with an endogenous, non-random response process. Domain judgment about *how the data came to exist* remains the researcher's job.

5 Transparency, Disclosure, and Research Norms

As agents enter the research pipeline, the profession is working out norms for disclosure. Open questions raised in the series include:

- When and how should authors disclose AI assistance in a paper?
- How do journals and "journal cartels" adapt their policies to AI-assisted analysis and writing?
- Where does accountability sit when an agent wrote the code? (Answer in practice: with the human authors, who must understand and stand behind every result.)
- How do we preserve scientific credibility against a rising tide of low-effort, AI-generated "slop"?

The throughline is that AI augments but does not absolve: the researcher remains responsible for correctness, interpretation, and honesty.

6 Open Research on the Economics of AI

Beyond using the tools, the newsletter tracks AI as an *object* of study: the autonomy and behavior of AI agents themselves, the macroeconomics of hundreds of billions in AI capital expenditure and whether it can earn a return, the labor-market and political effects of AI exposure, and the security implications of agents that can exploit or patch vulnerabilities faster than humans. These are active frontiers for empirical economists.

7 Takeaways

1. Coding agents like Claude Code make research *execution* cheap, which raises the premium on clear design and rigorous verification.
2. Discipline—checklists, simulation, externalized memory, version control—is what separates productive use from confident error.
3. The human researcher retains ownership of identification, judgment, accountability, and disclosure.

Source material: Scott Cunningham, *Scott's Mixtape Substack* (https://causalinf.substack.com/), including the "Claude Code" series for quantitative social scientists and the latest "closing tabs" newsletter from Lake Maggiore (June 14, 2026). This document is an independent conceptual synthesis.